

Scheduling and ACPI

Overview

In this project you will implement an operating system scheduler for a heterogeneous multicore processor that consists of big and small cores. The scheduler will be implemented in the context of a simulator with a random workload that offers opportunities for experimenting with different scheduling policies. The cores support the ACPI standard, thus allowing you great latitude for picking the right C-states and P-states, and even turning off individual cores or the entire chip when not needed.

You will be given the object files of the simulator, along with header files that define the programmatic types and function interfaces. Your task is to modify the file `Scheduler.cpp` with your implementation to minimize the overall energy consumption during a run. The file contains a skeleton scheduler to show you how everything fits together. You will need to modify that scheduler with your own, and hopefully (if not surely) reduce the amount of energy of the entire run. You may not change the `.h` files.

The output of your work will be:

- The modified code for `Scheduler.cpp` and any other files that you wish to add.
- A short report that describes your scheduling algorithm and the results that you obtained.
- For the top four performing projects in the class, an in-class short presentation showing the algorithm(s) and overall implementation effort. The top four performing projects will receive an additional bonus.

You will need at the beginning to spend some time reading through this document and familiarize yourself with the simulator interface. You should not need to know the inner workings of the simulator. Some approximations were made to promote robustness, but that should be invisible to you. What you need to focus on is:

- The choice of the scheduling algorithm that you will implement.
- Tracking the states of the processes and cores the information that the simulator provides during a run, and actuating based on this information to promote energy saving and/or performance optimizations.
- Testing and reporting the energy and performance results.

Grading:

The project will be graded for a total of 10 points. The grade distribution is as follows:

- Submission of code: 7 points.
- Submission of report: 3 points.

Bonuses:

- Top four performers: *up to* 4 bonus points.

Bonus points will go to offset points missed in the exams. They may not count toward your project grade.

Do not try to game the simulator! Different input sequence may be used during grading to ensure the robustness of your implementation. Try to have your solution independent of any specific input set.

Logistics

Deadlines:

- Your code is due on October 9th at 5:00PM. This is a FIRM deadline.
- Your report is due on October 11th. This is also a FIRM deadline.

The amount of time allocated to the project exceeds the time that is needed. The purpose here is to give you enough flexibility to manage your effort. Please be aware that waiting until the last possible minute to start is likely to cause problems. Pace yourself and start early.

Groups:

The project can be implemented in groups of two or three Please sign up for groups on Canvas in the appropriate group pages.

Extensions:

No extensions will be given. There is plenty of time for you to plan. Do not wait until the last minute. Exceptions will be made when, in the instructor's judgement, you are facing a situation of documented force majeure in your life.

Coding:

The simulator is written in C++. It will be most straightforward to implement your work in C++. However, you can use any programming language that you wish, provided that you are responsible for implementing the translation between your favorite language and C++. And yes, any programming language will be acceptable (even Python!). However, you will be responsible for all the additional logistics if you do not implement your project in C++. For health-related reason, the instructor will not be able to help you debug your code if written in a language that he doesn't like (e.g., Python).

Internet Tools and LLMs:

The use of Internet tools such as ChatGPT, Co-Pilot, etc. is permitted. However, you must make the necessary attributions in your final report and in the code that you borrow. Be a scholar and do not plagiarize the fruits of labor of these tools. LLMs are people too, or at least they are behaving like people, aren't they? Also, keep in mind that they tend to hallucinate and pretend that they know it all. They don't.

Bugs and Reporting:

The simulator has been tested. You should not have problems in general. However, the instructor is only human, and humans do not write perfect code. If you suspect a strange behavior, you need to alert the instructor directly and immediately. Every effort will be made to fix problems promptly. From time to time, we may refresh the simulator object files to reflect bug fixes and other changes. You may also report a need for a specific feature that the simulator doesn't provide. Every effort will be made to evaluate your

proposed need and respond appropriately given the constraints of time and available bandwidth and potential impact of changing the interface file on other groups.

Project files

The following files are available in GitHub at the following URI:

https://github.com/UTCS-350M/Project_1

The repository contains the following files:

Object files: libsim.so.

Source file: scheduler.cpp (this is the file that you need to modify to implement your algorithm(s).

Include files: interfaces.h, sim_types.h, and scheduler.hpp.

Build files: Makefile.

To access the project files, clone the repository into your hub. You must use the laboratory machines for this purpose. Read the README file for instructions on how to build the project files and how to run the simulator.

Getting Started

The following approach is recommended:

1. Access the project files and build the simulator. Use the supplied Scheduler.cpp to play with the simulator and get familiar with it.
2. Change some of the parameters in the Scheduler.cpp to see the differences that various energy management policies will produce.
3. Identify the algorithm(s) that you would like to implement.
4. Implement the algorithm(s) separately and use unit testing to ensure your code is working.
5. Integrate with the simulator files.
6. Write the report.

Report Structure

The report should be short. It should identify and describe the algorithm(s) that you are using. Then, it needs to show the running time of the entire simulation and energy consumption. Please do not exceed two pages.

Simulation Run

The simulator creates processes and invokes the scheduler to decide where to place them (on which core). As processes complete, the scheduler deallocates them and may perform activities that aim at reducing the CPU's overall energy consumption. The CPU consists of eight cores: Four "big cores" numbered 0 through 3, and four "small cores" numbered 4 through 7. The energy consumption of the small cores is roughly about one half of the big core, and the performance is about 60%. The cores can be put in different C-states, and at

C0, they can be put at different performance levels (P-states). The meaning of those states is as follows:

C0: A core is operating normally, running instructions.

C1, C2: Different modes of idling. C2 consumes less energy but is slower in restart.

C3, C4: Different modes of low voltage, again with C4 consuming less energy but being slower in restart.

C5: Not used.

C6: Core is turned off and does not consume any energy. It will take milliseconds to resume.

C7: The entire chip is turned off and will wake up only on interrupts. It consumes negligible amounts of energy but requires several seconds to restart.

The performance levels are defined by the P-states:

P0: Represents the fastest, sustainable speed, at the highest level of dynamic energy.

P1—P4: Represent a gradient in dynamic energy consumption and performance, each linearly reduced in terms of frequency with a higher rate of dynamic energy consumption.

Your task

You will need to implement the following components:

1. The scheduler infrastructure. This includes all queues and algorithms that implement the scheduling policies and how to set the processes to reduce energy consumption.
2. Document the algorithm in writing as part of your submission of this project.

Simulation Interface

The simulation presents an interface to enable the implementation of algorithms in the Scheduler module. The following is a description of the interface grouped by functionality.

Scheduling Management Interface

Down Calls:

void LoadContext(ProcessId_t pid, CPUId_t core)

This function attaches the context of process pid to the given core. This must be done before running the process on the core. An exception may occur if the input values are illegal. The core must be in C-state C1, or else an exception may be generated.

void SaveContext(ProcessId_t pid, CPUId_t core)

This function saves the state of the current process and should be performed before the process is pre-empted from the processor. LoadContext and SaveContext can be used to implement context switching. SaveContext must be called whenever a process is to be removed from a core, or bugs will occur. The core must be in C-state C0, or else an exception may be generated.

void RunCore(CPUId_t cpu_id)

Dispatches the core to run the process that was used in the last “LoadContext” call. Must be called to start the run. This is equivalent to calling RTI in real systems.

Up Calls:

void CreateProcess(ProcessId_t pid)

This upcall is informative. It informs the scheduler that the process whose identifier is pid has been created. The scheduler should incorporate the process into its queues and start scheduling it.

void ExitProcess(ProcessId_t pid)

This upcall is informative. It informs the scheduler that the process whose identifier is pid has finished execution. The scheduler should remove the process from its queues and stop scheduling it.

void TimerInterrupt(Time_t now)

This upcall is the heart of the simulation. It alerts the scheduler that there is a timer interrupt and invokes it. The scheduler should perform its scheduling decisions here, including performing any queue adjustment and context switching as part of its scheduling algorithm.

Energy Management Interface

Down Calls

void SetPState(CPUId_t core_id, PState_t p_state)

This function sets the P-state of a core to the value `p_state`. Acceptable values for `core_id` are 0 to 7, and P0 to P4 for `p_state`. An exception may be raised if the values are out of range. You can call this function any time on any core. If the C-state is not C0, the change will not have an effect. Most P-state changes are instantaneous.

void SetCState(CPUId_t core_id, CState_t c_state)

This function sets the C-state of a core to the value `c_state`. Acceptable values for `core_id` are 0 to 7 and C0 to C7 for `c_state`. C5 is not used so don't bother with this. Transitions from {C0, C1, C2} to {C3, C4} and vice versa take time. The system reports when the transition to {C0, C1, C2} is complete. The system does not report when the transition to {C3, C4} is complete as it is not needed. Similarly, the transition to {C6, C7} takes time and its completion is reported in an upcall. Exceptions may be raised if the input to the function is not valid. Note that setting one core to C7 will shut down the entire chip, which means that all cores will move to C7 state. Do not do this while one core is still running at C0 (has a process running). Also, transitioning from C0 to any of the lower energy state is problematic in general because it means that you do not save the context of the running process before effectively stopping computation. This will create bugs and should be avoided.

double GetTotalEnergyConsumed()

This function can be called any time to display the energy that was consumed thus by the eight cores.

Upcalls

void CStateTransitionComplete(CPUId_t core_id){

Informs the scheduler that the core has completed a transition from either {C3, C4} or {C6, C7} to C1.

Exception Handling and Error Reporting

Exception handling is used to alert the programmer to errors. The default action for exception handling is to write a message and stop the simulation. If you wish to change the default handling, you can capture the following exception signature in a try/catch structure as usual:

```
extern void      ThrowException(string err_msg);
extern void      ThrowException(string err_msg, string further_input);
extern void      ThrowException(string err_msg, unsigned further_input);
```

The exception handler has the following syntax:

```
throw(runtime_error(err_msg + further_input))
catch(const runtime_error & error)
```

In addition, the simulator can run with the `-v [0-4]` flag. This will turn on the audit trail. It is quite verbose and will generate a huge volume at level 4 that describes the innerworkings

of the simulator (and will significantly slow down the simulation). Levels 0 to 3 are mostly reserved for you. You can use the following function if you would like to add a message to the audit trail. The audit trail will be very useful for debugging, especially if it needs to involve the instructor.

```
extern void      SimOutput(string msg, unsigned verbose_level);
```

Currently, the flag is disabled. However, you can set up the `verbose_level` and call `SimOutput` to whatever value you like. Beware that you will be inundated with print statements.

Simulation completion

When the simulator completes its function, the last function it calls is the upcall `SimulationComplete()`. In this function, you should report the results of the simulation (energy consume and total running time. The code provided already performs this function.

Units of measurements

To facilitate the innerworkings of the simulator, the following units are used:

- Time in microseconds
- Energy in kwh

Appendix: Simulation Structure

Strictly speaking, this section is not mandatory and is provided here to give you a glimpse at how the simulator works internally. The structure below shows the different components:

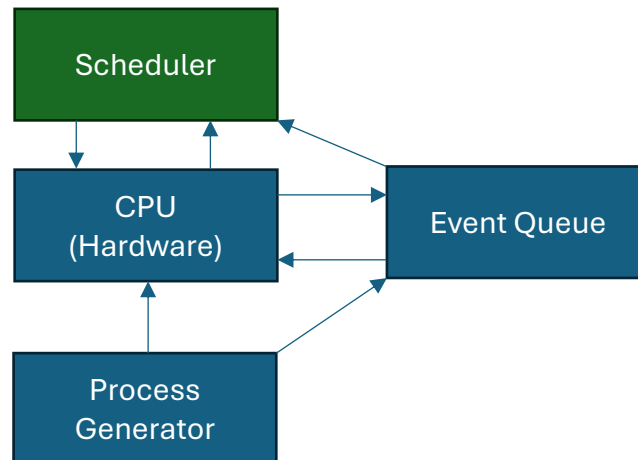


Figure 1. Simulator structure

The heart of the simulator is an event queue sorted by time. This implements a conventional event-driven simulation. Events on the queue are added by the other components, and after the initialization is complete, the simulator consists of a loop that takes the events in time order and calls the corresponding functions in the other modules.

The simulator continues in the simulation loop until no more events are in the queue, at which point the simulation stops. It maintains a clock that advances with the occurrence of each event. For robustness, it is not possible to have a time warp (insertion of an event at a time less than the current simulation time), and it is not possible to strike out events. The current time of the simulation is maintained by the function “Now()”, which is implemented by the event queue module.

The “process generator” module is responsible for generating each process, including the corresponding arrival event, and how much time the process needs in terms of processing on the CPU.

The “CPU” module contains the bulk of the simulator functions. These emulate the processing and energy consumption of the CPU cores. The module monitors the processes in the system and as long as they have not finished, it will continue to generate timer events. The “CPU” module is invoked by the timer events. The timer handler is where the processes advance in their executions on the various simulated cores. The energy consumed is also updated in the timer handler.

The general implementation of these components uses an object-oriented approach with a flat object hierarchy. The interfaces between the modules use regular function calls and not

object invocations. This is done to simplify the implementation and avoids the problematic issues that result from circular inclusions of include files (a famous problem that C++ has).

GOOD LUCK!

START EARLY